

Міністерство освіти і науки України
Чернівецький національний університет імені Юрія Федьковича

Програмного забезпечення комп'ютерних систем

(повна назва кафедри, циклової комісії)

КУРСОВИЙ ПРОЄКТ: СТРУКТУРИ ДАНИХ, АЛГОРИТМИ ТА ОБ'ЄКТНО-ОРІЄНТОВАНА ПАРАДИГМА

на тему: Система автоматизації складу товарів

здобувач вищої освіти II курсу 243 групи
спеціальності 121 «Інженерія програмного
забезпечення»
першого (бакалаврського) рівня вищої освіти
Кирилюк М.О.

Дата захисту «_____» _____ 20__ р.

Оцінка:

за національною шкалою _____
(словами)

кількість балів _____
(цифра)

за шкалою ECTS _____
(літера)

Чернівці, 2025

ЧЕРНІВЕЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ЮРІЯ ФЕДЬКОВИЧА

Навчально-науковий інститут фізико-технічних та комп'ютерних наук
Кафедра програмного забезпечення комп'ютерних систем

ЗАВДАННЯ
на курсовий проєкт
здобувачу вищої освіти
першого (бакалаврського) рівня вищої освіти
Кирилюк Максим Олександрович

1. Тема проєкту: Система автоматизації складу товарів.

2. Вихідні дані до проєкту:

- Створити базу даних – CSV, в який будуть вноситись дані про товари;
- Передбачити ввід та вивід даних через консоль;
- Розробити консольне меню, для виконання функцій програми (додавання, редагування, видалення, перегляд);
- Передбачити сортування та фільтрацію автомобілів з бази даних;
- Передбачити перевірку коректності вводу даних та обробку виключень.

3. Зміст розрахунково-пояснювальної записки (перелік питань, які необхідно розробити):

- описати загальні вимоги до програми;
- описати призначення та область застосування;
- описати модулі проекту та алгоритми виконання;
- описати методи програми;
- описати користувацький інтерфейс.

4. Перелік графічного матеріалу (з точним зазначенням обов'язкових креслень):

- блок-схеми роботи з програмою;
- скріншоти роботи з програмою.

Керівники проекту

(підпис керівника)

Валь О.О.

(підпис керівника)

Дяконенко Б.В.

(підпис керівника)

Комісарчук В.В.

Завдання взяв до виконання

(підпис студента)

Кирилюк М.О

РЕФЕРАТ

У курсовому проєкті розроблено систему управління базою товарних записів магазину для ноутбуків або персональних комп'ютерів, що працюють під керуванням операційної системи Windows.

Програмне забезпечення розраховане на користувачів, які не мають спеціальної комп'ютерної підготовки. Досвід роботи згодом не вимагається.

Область застосування – програмне забезпечення для автоматизації процесів обліку, контролю, та інвентаризації товарів на складі. Система забезпечує ідентифікацію користувача за ролями (Адміністратор та Касир) та розмежування доступу до функціоналу (CRUD операції, сортування, фільтрація, інвентаризація).

Розробка реалізована засобами середовища CLion на мові C++. Використання CLion та системи збірки CMake дозволяє створювати крос-платформний консольний додаток, що є зручним для розробки та подальшого використання на різних операційних системах.

Дана розробка у майбутньому може бути розширена із додаванням нового функціоналу і видозміненою логікою обробки.

Курсовий проєкт містить: 73 с., 25 рис., 5 табл., 1 додаток, 6 джерел.

ОПЕРАЦІЙНА СИСТЕМА, НОУТБУКИ, CLION, C++, КОНСОЛЬНИЙ ДОДАТОК.

SUMMARY

The course project developed a Store Inventory Management System for laptops or personal computers running Windows and Linux operating systems.

The software is designed for users who do not have specialized computer training. No prior experience with the application is required.

The Field of Application is software for automating inventory accounting, control, and stocktaking processes in a warehouse. The system ensures user identification by roles (Administrator and Cashier) and access differentiation to functionality (CRUD operations, sorting, filtering, inventory). The development is implemented using the Visual studio environment in C++. This environment is convenient to use for quick and high-quality creation of applications on OC Windows.

The development is implemented using the C++ language within the CLion environment. Utilizing CLion and the CMake build system allows for the creation of a cross-platform Console Application, which is convenient for development and subsequent use on various operating systems.

This development can be expanded in the future by adding new functionality and modified processing logic.

The course project contains: 73 pages, 25 figures, 5 tables, 1 appendix, 6 references.

*OPERATING SYSTEM, LAPTOPS, COMPUTERS, CLION, C++,
CONSOLE APP, INVENTORY MANAGEMENT SYSTEM, WAREHOUSE
ACCOUNTING, STOCKTAKING .*

ЗМІСТ

ЗМІСТ.....	5
ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ	6
1 АРХІТЕКТУРА ТА ФУНКЦІОНАЛЬНІ ПОКАЗНИКИ	8
1.1 ЗАГАЛЬНІ ВИМОГИ ДО ПРОГРАМИ	8
1.1.1 Вимоги до консольного інтерфейсу користувача	8
1.1.2 Вимоги до архітектури програми	8
1.1.3 Вимоги до функціональності програми	9
1.2 ПРИЗНАЧЕННЯ ТА ОБЛАСТЬ ЗАСТОСУВАННЯ	9
1.3 ФУНКЦІОНАЛЬНІ ВИМОГИ	9
1.4 Опис сценаріїв використання програми.....	11
2 ОПИС ПРОГРАМИ	16
2.1 СТРУКТУРА ПРОГРАМИ	16
2.1.1 Модулі програми	16
2.1.2 Алгоритми роботи програми.....	17
2.2 ОПИС МЕТОДІВ ПРОГРАМИ.....	22
2.3 ПРОГРАМНІ ЗАСОБИ.....	26
и2.4 ОПИС КОРИСТУВАЦЬКОГО ІНТЕРФЕЙСУ.....	26
ВИСНОВКИ.....	36
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	37
ДОДАТКИ.....	38
ДОДАТОК А. СКРОЛІНГ ПРОГРАМИ.....	38
ПРОГРАМНИЙ МОДУЛЬ "Main.cpp"	38
ПРОГРАМНИЙ МОДУЛЬ "Admin"	40
Вміст "Admin.h"	40
Вміст "Admin.cpp"	41
ПРОГРАМНИЙ МОДУЛЬ "AutoManager"	42

Вміст "AutoManager.h"	42
Вміст "AutoManager.cpp"	43
ПРОГРАМНИЙ МОДУЛЬ "BasicUser"	48
Вміст "BasicUser.h"	48
Вміст "BasicUser.cpp"	49
ПРОГРАМНИЙ МОДУЛЬ "Configuration"	50
Вміст "Configuration.h"	50
Вміст "Configuration.cpp"	51
ПРОГРАМНИЙ МОДУЛЬ "IPrintable"	52
Вміст "IPrintable.h"	52
ПРОГРАМНИЙ МОДУЛЬ "Logger"	53
Вміст "Logger.h"	53
ПРОГРАМНИЙ МОДУЛЬ "Product"	54
Вміст "Product.h"	54
ПРОГРАМНИЙ МОДУЛЬ "Receip"	55
Вміст "Receip.h"	55
Вміст "Receip.cpp"	56
ПРОГРАМНИЙ МОДУЛЬ "SeleProcessor"	58
Вміст "SeleProcessor.h"	58
Вміст "SeleProcessor.cpp"	59
ПРОГРАМНИЙ МОДУЛЬ "StoreItem"	61
Вміст "StoreItem.h"	61
Вміст "StoreItem.cpp"	62
ПРОГРАМНИЙ МОДУЛЬ "StoreManager"	63
Вміст "StoreManager.h"	63
Вміст "StoreManager.cpp"	64
ПРОГРАМНИЙ МОДУЛЬ "User"	71
Вміст "User.h"	71
Вміст "User.cpp"	72

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

ОС – операційна система.

ПЗ – програмне забезпечення.

ID – ідентифікатор.

CRUD — це скорочення від англійських слів Create, Read, Update, Delete.

БД - База Даних.

1) АРХІТЕКТУРА ТА ФУНКЦІОНАЛЬНІ ПОКАЗНИКИ

1.1 Загальні вимоги до програми

Вимоги до консольного інтерфейсу користувача

1. Робоча мова інтерфейсу – українська.
2. Використання термінів, зрозумілих користувачеві.
3. Навігаційна панель (меню), яка забезпечує перегляд, редагування, додавання та зчитування даних.
4. «Посібник користувача», що містить інструкції з використання програми.
5. Повідомлення про некоректно введені дані.

Вимоги до архітектури програми

- 1) Використання класів, класів-нащадків та абстрактних класів, інтерфейсного класу які реалізують:
 - збереження даних у файли та зчитування даних з файлів.
 - методи вводу та виводу даних.
- 2) Стійкість програми:
 - програма повинна не втрачати працездатності при будь-яких діях користувача;
 - інформація, що вводиться, скрізь, де це можливо, піддається логічному забезпеченню цілісності даних;
 - при будь-яких діях користувача не повинні втрачатись дані або їх цілісність.

Вимоги до функціональності програми

1. База даних – це текстовий файл.
2. Збереження даних у БД, їх зчитування, зміна та видалення.
3. Обов'язкова перевірка коректності вводу даних та обробка виключень.
4. Сортування, фільтрація та пошук даних по самостійно обраному ключу.
5. Наявність посібника користувача, який ознайомить користувача з принципом взаємодії з програмою.
6. Можливість пошуку об'єктів.

1.2 Призначення та область застосування

Мета роботи полягає у розробці системи управління базою автомобілів для комп'ютерів, які працюють під керуванням ОС Windows та Linux.

Програмне забезпечення покращує управління товарами, а саме: додавання, редагування, видалення та отримання даних про товар за заданими фільтрами. А також робить зручним пошук товарів за певними параметрами.

Область застосування – програмне забезпечення для ноутбуків або персональних комп'ютерів, що працюють на операційній системі Windows.

1.3 Функціональні вимоги

До програмного забезпечення висуваються такі функціональні вимоги:

- 1) Вхідні дані подаються користувачем методом введення із клавіатури у консоль.
- 2) База даних – це CSV файл, тому передбачити:
 - збереження даних у файл;
 - зчитування даних з файлу.
- 3) Консольне меню повинне забезпечувати наступні можливості:
 - перегляд даних;
 - редагування даних;
 - додавання даних;
 - видалення даних;
 - перегляд посібника користувача.
- 4) обов'язкова перевірка коректності вводу даних та обробка виключень.

1.4 Опис сценаріїв використання програми

Сценарій 1: Ініціалізація системи	
Передумова: Файли даних існують. Програма запускається.	
Основний сценарій	
Крок 1	Система успішно зчитує дані (товари, користувачі).
Крок 2	Створюються об'єкти в пам'яті.
Крок 3	Відображено меню входу.
Альтернативний сценарій	
Крок 1	Файл даних пошкоджений або відсутній.
Крок 2	Система ініціалізує порожню колекцію.
Крок 3	Виводиться повідомлення про ризик роботи без збережених даних.

Сценарій 2: Аутентифікація користувача (адмін)	
Передумова: Користувач вводить коректні дані.	
Основний сценарій	
Крок 1	Користувач вводить логін/пароль Адміністратора.
Крок 2	Система ідентифікує роль Адміністратора.
Крок 3	Відкривається повне меню.
Альтернативний сценарій	
Крок 1	Користувач вводить невірний пароль.
Крок 2	Система виводить “Невірний пароль”.
Крок 3	Залишається можливість повторного вводу.

Сценарій 3: Додавання товарів	
Передумова: Обрано пункт 1.	
Основний сценарій	
Крок 1	Користувач вводить унікальний ID та всі коректні атрибути
Крок 2	Перевірка унікальності ID та числового формату.
Крок 3	Об'єкт додається до колекції, виводиться повідомлення про успіх.
Альтернативний сценарій	
Крок 1	Користувач вводить ID, який вже існує.
Крок 2	Система виявляє дублікат ID.
Крок 3	Виводиться Помилка: Товар з таким ID вже існує.

Сценарій 4: Редагування товару	
Передумова: Обрано пункт 2	
Основний сценарій	
Крок 1	Введено існуючий ID.
Крок 2	Користувач обирає поле та вводить нове коректне значення.
Крок 3	Об'єкт оновлюється в пам'яті. Виводиться "Товар успішно оновлено"
Альтернативний сценарій	
Крок 1	Введено неіснуючий ID.
Крок 2	Система не знаходить об'єкт.
Крок 3	Виводиться повідомлення "Товар не знайдено"

Сценарій 5: Видалення товару	
Передумова: Обрано пункт 3	
Основний сценарій	
Крок 1	Введено товар.
Крок 2	Користувач підтверджує операцію.
Крок 3	Об'єкт видаляється з колекції, виводиться "Товар успішно видалено".
Альтернативний сценарій	
Крок 1	Введено товар.
Крок 2	Користувач відмовляється від видалення
Крок 3	Операція скасовується. Об'єкт залишається у колекції.

Сценарій 6: Перегляд товару	
Передумова: Обрано пункт 4	
Основний сценарій	
Крок 1	Система перебирає всі об'єкти у колекції.
Крок 2	Вивід Дані кожного товару послідовно виводяться на екран.
Альтернативний сценарій	
Крок 1	Колекція товарів порожня.
Крок 2	Система виводить: "База даних порожня."

Сценарій 7: Пошук товару	
Передумова: Обрано пункт 5	
Основний сценарій	
Крок 1	Обрано критерій назва та введено ключове слово.
Крок 2	Система фільтрує колекцію, застосовуючи пошук підрядка.
Крок 3	Виводиться список знайдених товарів.
Альтернативний сценарій	
Крок 1	Ключове слово відсутнє в базі.
Крок 2	Система завершує пошук без результату.
Крок 3	Виводиться “Співпадіння не знайдено”.

Сценарій 8: Сортування товару	
Передумова: Обрано пункт 6	
Основний сценарій	
Крок 1	Обрано критерій ціна та напрямок за спаданням.
Крок 2	Виконується сортування колекції за вказаним полем.
Крок 3	Відсортований список відображається на консолі
Альтернативний сценарій	
Крок 1	Введено некоректний код напрямку сортування
Крок 2	Система не може визначити напрямок.
Крок 3	Виводиться: “Некоректний вибір. Повторіть ввід.”

Сценарій 9: Фільтрація товару	
Передумова: Обрано пункт 7	
Основний сценарій	
Крок 1	Обрано критерій (Ціновий діапазон) та введено коректні min/max.
Крок 2	Створюється новий список об'єктів, що відповідають умові.
Крок 3	Відображається відфільтрований список.
Альтернативний сценарій	
Крок 1	Введено min значення більше ніж max.
Крок 2	Система виявляє логічну помилку.
Крок 3	Виводиться: "Помилка: Нелогічний діапазон."

Сценарій 10: Інвентаризація товару	
Передумова: Обрано пункт 8	
Основний сценарій	
Крок 1	Система перебирає колекцію, порівнюючи Залишок з Критичним Мінімумом.
Крок 2	Створюється звіт про дефіцитні позиції
Крок 3	Звіт відображається.
Альтернативний сценарій	
Крок 1	Усі залишки в нормі.
Крок 2	Звіт не генерується.
Крок 3	Система виводить: "Інвентаризація завершена. Усі залишки в нормі."

Сценарій 11: Керування обліковими записами	
Передумова: Обрано пункт 9, виводиться керування користувачами обираємо пункт 2	
Основний сценарій	
Крок 1	Обрано "Створити".
Крок 2	Адміністратор вводить новий логін/пароль та роль Касир.
Крок 3	Обліковий запис додано.
Альтернативний сценарій	
Крок 1	Вхід виконано як Касир.
Крок 2	Система перевіряє права доступу
Крок 3	Виводиться: "Відмовлено в доступі. Недостатньо прав."

Сценарій 12: Аутентифікація користувача (юзер)	
Передумова: Користувач вводить коректні дані	
Основний сценарій	
Крок 1	Користувач вводить коректний логін та пароль.
Крок 2	Система ідентифікує роль Касир.
Крок 3	Надано доступ до обмеженого меню
Альтернативний сценарій	
Крок 1	Введено неіснуючий логін або пароль.
Крок 2	Система виводить "Користувача не знайдено" або "Невірний пароль".
Крок 3	Виводиться: "Відмовлено в доступі. Недостатньо прав."

Сценарій 13: Продаж товару	
Передумова: Обрано пункт 2	
Основний сценарій	
Крок 1	Введено ID та кількість (5).
Крок 2	Залишок оновлено (10→5).
Крок 3	Генерується Чек, запис про транзакцію зберігається.
Альтернативний сценарій	
Крок 1	Введено кількість (15), що перевищує залишок (10).
Крок 2	Система виявляє невідповідність.
Крок 3	Виводиться: "Помилка: Недостатня кількість товару. Транзакція скасована."

Сценарій 13: Перегляд чеків	
Передумова: Обрано пункт 3	
Основний сценарій	
Крок 1	Введено ID та кількість (5).
Крок 2	Залишок оновлено (10→5).
Крок 3	Генерується Чек, запис про транзакцію зберігається.
Альтернативний сценарій	
Крок 1	Введено кількість (15), що перевищує залишок (10).
Крок 2	Система виявляє невідповідність.
Крок 3	Виводиться: "Помилка: Недостатня кількість товару. Транзакція скасована."

2 ОПИС ПРОГРАМИ

2.1 Структура програми

Модулі програми

Робота розробленого програмного забезпечення реалізується наступними модулями:

1. `Product` – абстрактний клас що представляє загальну товарну одиницю.
2. `Car` – клас нащадок `Vehicle`, клас що представляє автомобіль.
3. `Configuration` – статичний клас . Він описує програми (шляхи до файлів `store_items.csv` та `users.txt`), а також містить допоміжні статичні методи, наприклад, для безпечного вводу даних (`GetSafeNumberInput()`) та управління логером.
4. `StoreManager` – клас, який керує колекцією об'єктів `StoreItem`. Реалізує всю бізнес-логіку програми: CRUD-операції, завантаження/збереження, пошук, сортування, фільтрацію та аналітику.
5. `AuthManager` – клас для управління авторизацією та обліковими записами користувачів.
6. `IPrintable` – абстрактний клас-інтерфейс. Всі класи, що наслідують його зобов'язані представити реалізацію методів `ToString()` and `ToCSVLine()`

1.1.1 Алгоритми роботи програми

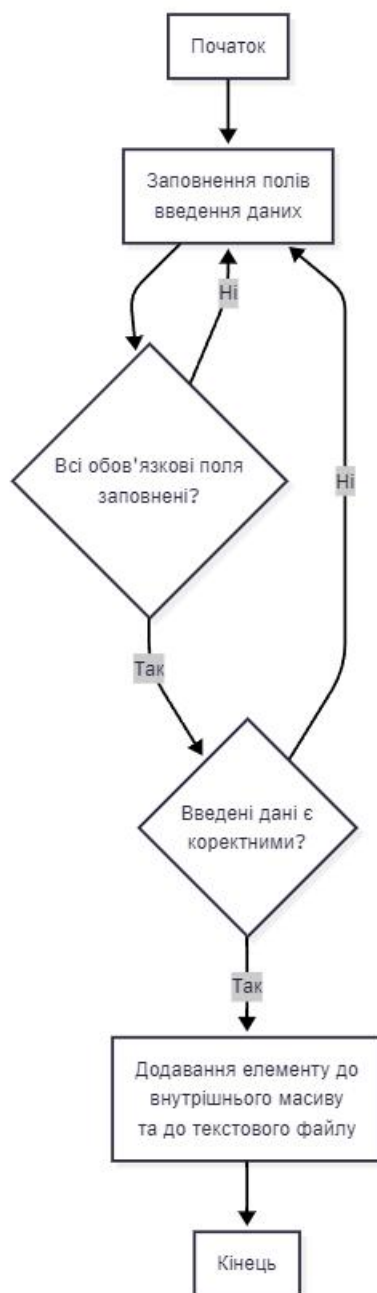


Рисунок 1 – Узагальнена блок-схема алгоритму додавання об'єкту

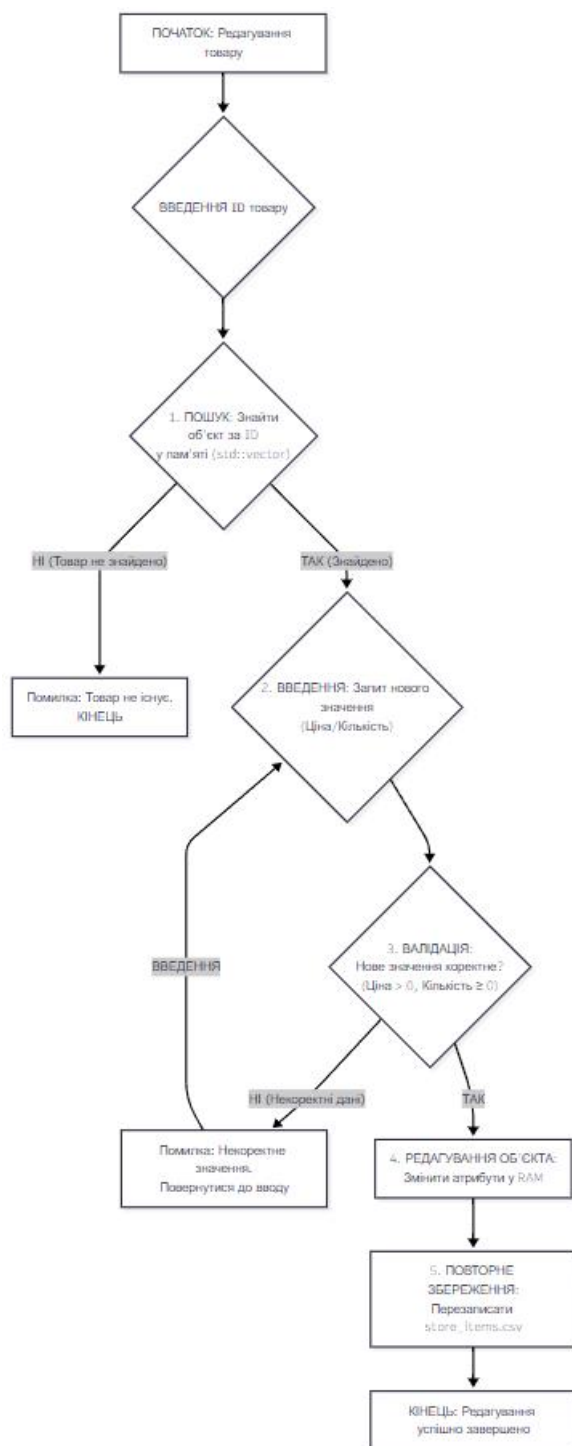


Рисунок 2 – Узагальнена блок-схема алгоритму редагування об'єкту

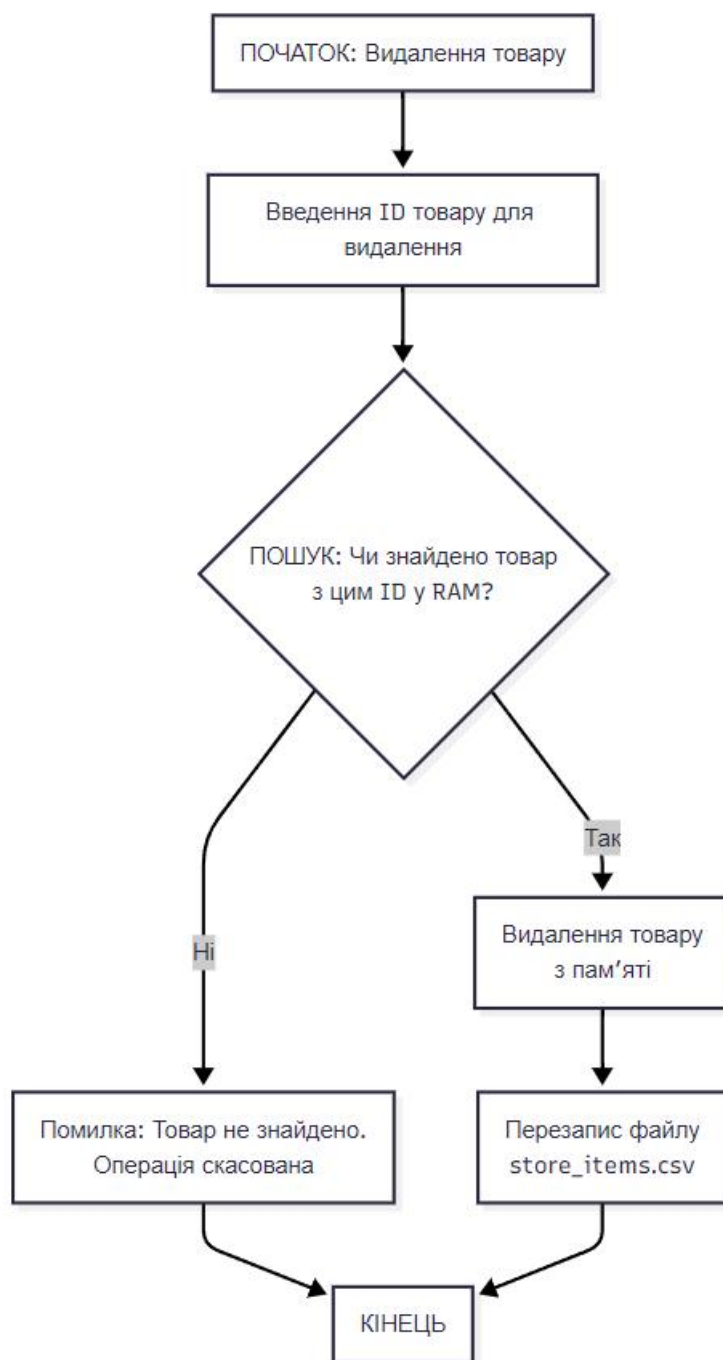


Рисунок 3 – Узагальнена блок-схема алгоритму видалення об'єкта

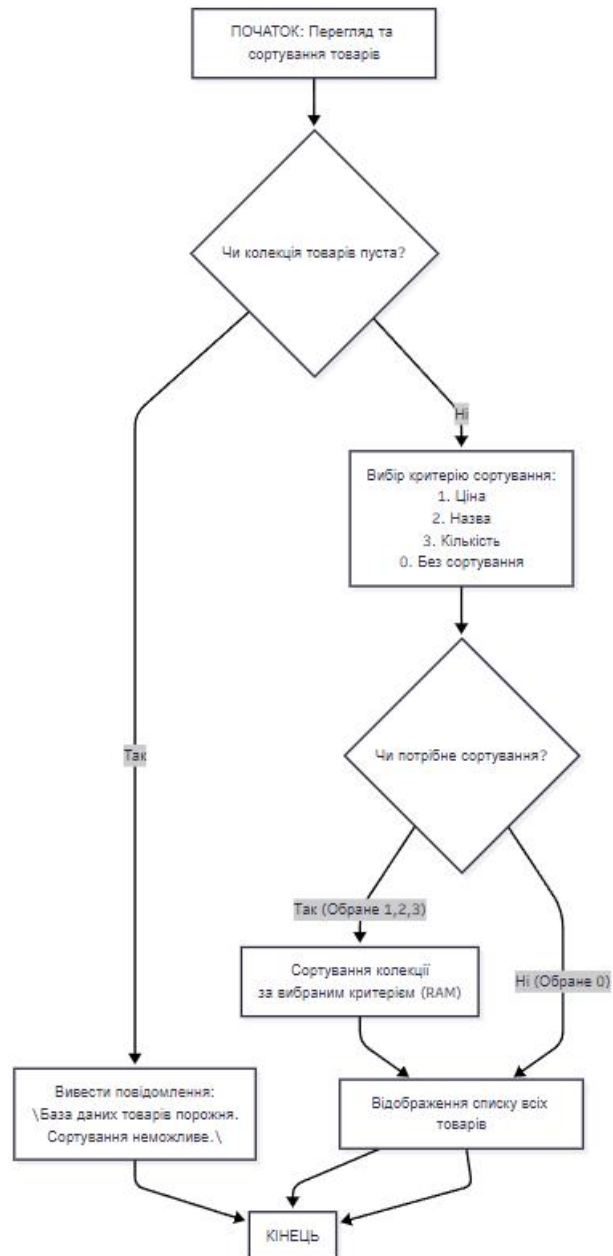


Рисунок 4 – Узагальнена блок-схема алгоритму перегляду доданих об'єктів та їх сортування

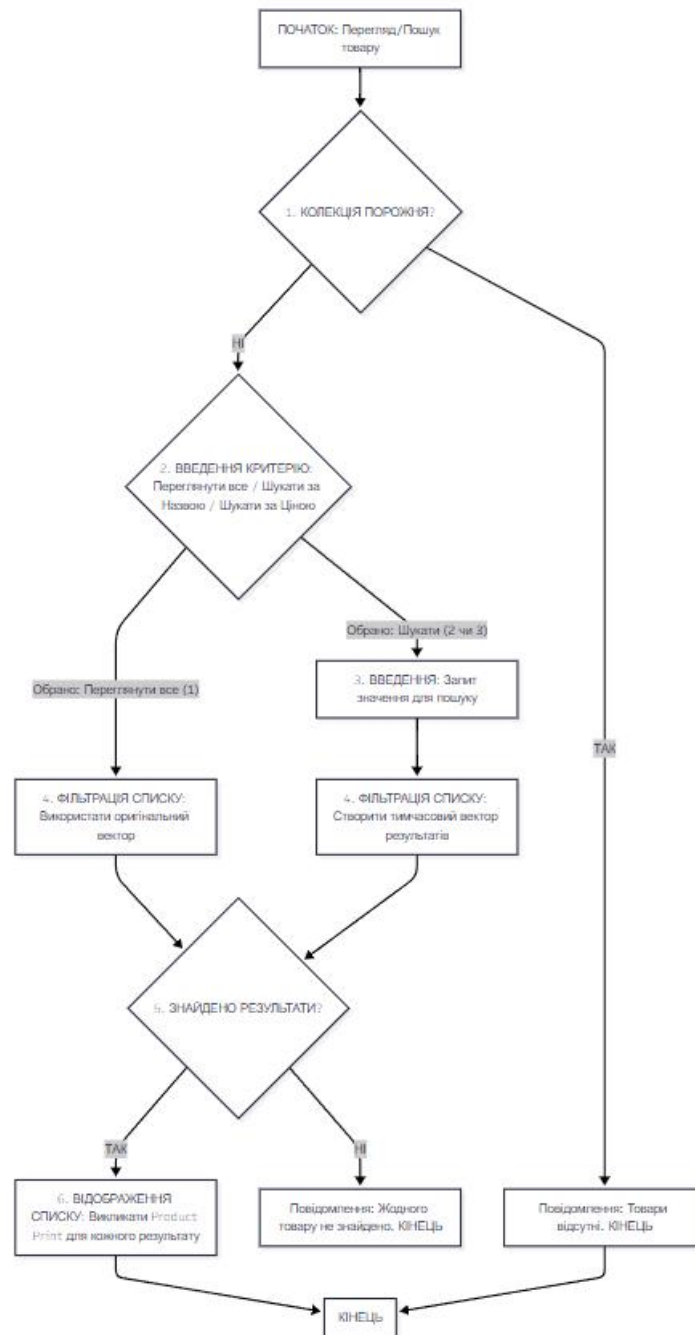


Рисунок 5 – Узагальнена блок-схема алгоритму перегляду доданих об’єктів та їх пошуку

2.2 Опис методів програми

Список методів класу `IPrintable` та їх опис наведено в табл.1.

Таблиця 1 – Основні методи класу `IPrintable`

№	Назва методу	Короткий опис
1	<i>ToString()</i>	Чисто віртуальний. Повертає повну інформацію про об'єкт у вигляді форматovanого рядка для виведення в консоль.
2	<i>ToCSVLine()</i>	Чисто віртуальний. Формує рядок даних об'єкта у форматі CSV для запису у файл.

Список методів класу `Product` та їх опис наведено в табл.2.

Таблиця 2 – Основні методи класу `Product`

№	Назва методу	Короткий опис
1	<i>SetPrice()</i>	Встановлює ціну товару з обов'язковою валідацією.
2	<i>GetPrice()</i>	попфторяє поточну ціну товару.
3	<i>DisplayInfo()</i>	Чисто віртуальний. Виводить інформацію про товар у консоль. Реалізується у <code>StoreItem</code> .

Список методів класу *StoreItem* та їх опис наведено в табл.3.

Таблиця 3 – Основні методи класу *StoreItem*

№	Назва методу	Короткий опис
1	<i>ColculateValue()</i>	Обчислює сумарну вартість залишку цього товару: Ціна * Кількість.
2	<i>UpdateQuantity()</i>	Реєстрація надходження, збільшуючи поточну кількість на складі.

Список методів класу *User* та їх опис наведено в табл.4.

Таблиця 4 – Основні методи класу *User*

№	Назва методу	Короткий опис
1	<i>GetRole()</i>	Перевіряє роль користувача (наприклад, "admin" або "basic user").
2	<i>GetUsername()</i>	Повертає логін користувача.
3	<i>CheckPermissions()</i>	Чисто віртуальний. Використовується для визначення, які функції меню доступні користувачу.

Список методів класу StoreManager та їх опис наведено в табл.5.

Таблиця 5 – Основні методи класу StoreManager

№	Назва методу	Короткий опис
1	<i>LoadDate()</i> <i>/SaveDate()</i>	Відповідає за зчитування/запис усіх товарів із/до файлу store_items.csv.
2	<i>AddObject()</i>	Реалізує CRUD-C. Додає новий об'єкт StoreItem після валідації вводу.
3	<i>UpdateObject()</i>	Реалізує CRUD-U. Оновлює кількість (надходження) та інші поля існуючого товару.
4	<i>DeleteObject()</i>	Реалізує CRUD-D. Видаляє об'єкт зі складу (списання/уцінка).
5	<i>RunInventory()</i>	Обчислює та виводить загальну сумарну вартість усіх товарних залишків у системі.
6	<i>SortObjects()</i>	Сортує внутрішню колекцію товарів за обраним критерієм (назва, ціна, кількість).
7	<i>FileObjects()</i>	Фільтрує колекцію, наприклад, виводить товари із залишком менше N.

Список методів класу AuthManager та їх опис наведено в табл.6.

Таблиця 6 – Основні методи класу AuthManager

№	Назва методу	Короткий опис
1	<i>AuthenticateUser()</i>	Головний метод. Перевіряє логін/пароль. У разі успіху повертає об'єкт User (конкретно Admin або BasicUser).
2	<i>LoadUsers()/</i> <i>SaveUsers()</i>	Відповідає за зчитування/запис облікових записів із/до файлу users.txt.
3	<i>AddNewUser()</i>	Додає новий обліковий запис до системи (функція доступна тільки адміністратору).
4	<i>DeleteUser()</i>	Видаляє існуючий обліковий запис.

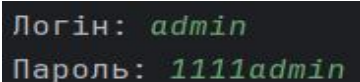
2.3 Програмні засоби

Розробка реалізована засобами середовища CLion на мові C++. Дане середовище є інтелектуальною та крос-платформною платформою для швидкого та якісного створення додатків.

Компанія JetBrains – це творець одного з провідних інтегрованих середовищ розробки (IDE), CLion, що надає глибоку підтримку C/C++ та повністю підтримує сучасні стандарти мови. Дане програмне забезпечення надає комплексний набір інструментів, що базується на інтелектуальному редакторі (включно з потужним відлагоджувачем, інструментами для рефакторингу, аналізу коду та юніт-тестування). Для автоматизації процесу побудови та забезпечення крос-платформності CLion використовує систему управління проєктами CMake як основний механізм. Така інтеграція спрощує збірку та адаптацію проєкту до різних операційних систем (Windows, Linux, macOS), що ідеально підходить для розробки вашого консольного додатку.

2.4 Опис користувацького інтерфейсу

Після запуску програми, в консоль виводиться меню авторизації користувача, авторизовуємось як адмін (рис. 5).



```
Логін: admin  
Пароль: 1111admin
```

Рисунок 5 - меню авторизації (адмін)

Після успішної авторизації, як адміністратор ми бачимо меню, яке складається з 10 пунктів (рис. 6). Адміністратор може додати, редагувати, видалити, переглянути, шукати, сортувати, фільтрувати товари, провести інвентаризацію залишків, керування обліковими записами, вийти з облікового запису.

```
=== Меню Адміністратора (АДМІН) ===  
1. Додати новий товар  
2. Редагувати товар  
3. Видалити товар (Уцінка/Списання)  
4. Переглянути всі товари  
5. Шукати товари  
6. Сортувати товари  
7. Фільтрувати товари  
8. Провести інвентаризацію залишків  
--- Адміністрування користувачів ---  
9. Керування обліковими записами  
0. Вийти з облікового запису
```

Рисунок 6 – меню адміністратора

Обравши пункт 1 головного меню, консоль виводить меню з додавання нового товару (рис. 7).

```
=== СТВОРЕННЯ НОВОГО ТОВАРУ ===  
Введіть назву товару: Картопля  
Картопля  
Введіть одиницю виміру (шт, кг, л): кг  
Введіть ціну одиниці: 100  
Введіть кількість: 20  
Введіть дату останнього завантаження (YYYYMMDD): 20240115
```

Рисунок 7 – меню з додавання нового товару

Обравши пункт 2 головного меню, консоль виводить редагування товару (рис. 8). Дозволяє відрегувати ціну, кількість, одиниці виміру.

```
=== РЕДАГУВАННЯ ТОВАРУ ===
Введіть назву товару для редагування: Картопля
Картопля
Що редагувати? 1. Ціна 2. Кількість 3. Одиниця виміру:
```

Рисунок 8 – редагування товару

Обравши пункт 3 головного меню, консоль виводить видалення товару (рис. 9).

```
=== ВИДАЛЕННЯ ТОВАРУ (Списання/Уцінка) ===
Введіть назву товару для видалення: Картопля
Картопля
// Товар 'Картопля' успішно списано/видалено з бази.
```

Рисунок 9 – видалення товару

Обравши пункт 4 головного меню, консоль виводить список товарів (рис. 10).

```
===== СПИСОК ТОВАРІВ =====
Назва          | Ціна | Залишок | Од. | Дата Завозу
-----|-----|-----|-----|-----
Хліб білий     | 25.50 | 50 | шт | 2024-01-15
Молоко 1л      | 45.00 | 30 | л  | 2024-01-20
Яйця курячі   | 3.50  | 120 | шт | 2024-01-18
Цукор білий   | 35.00 | 25 | кг | 2024-01-10i
Олія соняшникова | 55.00 | 20 | л  | 2024-01-22
Масло вершкове | 180.00 | 15 | кг | 2024-01-19
Сир твердий   | 220.00 | 12 | кг | 2024-01-21
Ковбаса варена | 150.00 | 18 | кг | 2024-01-20
Помідори      | 60.00 | 40 | кг | 2024-01-23
Огірки        | 45.00 | 35 | кг | 2024-01-23
Цибуля ріпчаста | 25.00 | 50 | кг | 2024-01-16
Морква        | 30.00 | 45 | кг | 2024-01-17
Капуста білокачанна | 15.00 | 60 | кг | 2024-01-18
ав            | 0.00  | 0  | мів | 123123
=====
```

Рисунок 10 – список товарів

Обравши пункт 5 головного меню, консоль виводить результат пошуку товару (рис. 11).

```
===== РЕЗУЛЬТАТИ ПОШУКУ за 'Морква' =====
--- Товар: Морква          | Ціна: 30.00 | Залишок: 45 кг | Завезено: 2024-01-17
```

Рисунок 11 – результат пошуку товару

Обравши пункт 6 головного меню, консоль виводить сортування за: (рис. 13). Дозволя відсортувати за назвою, ціною, кількістю. Обробивши пункт 6, консоль виводить відсортування за ціною (рис. 14).

```
Сортувати за: 1. Назва (ASC) 2. Ціна (ASC) 3. Кількість (DESC):
```

Рисунок 13 – сортувати за:

```
=====
|  Ціна | З
-----|--
3.50 |
20.00 |
| 25.00 |
25.50 |
| 30.00
35.00 |
| 45.00 |
| 45.00
| 55.00
60.00 |
150.00 |
180.00 |
220.00 |
```

Рисунок 14 – відсортовано

Обравши пункт 7 головного меню, консоль виводить товари залишок менший за: (рис. 15). Відфільтрований товар (рис. 16).

Показати товари, у яких залишок менший за:

Рисунок 15 – товари залишок менший за:

```
===== ФІЛЬТР: ЗАЛИШОК < 120 =====  
--- Товар: Хліб білий | Ціна: 25.50 | Залишок: 50 шт | Завезено: 2024-01-15  
--- Товар: Молоко 1л | Ціна: 45.00 | Залишок: 30 л | Завезено: 2024-01-20  
--- Товар: Цукор білий | Ціна: 35.00 | Залишок: 25 кг | Завезено: 2024-01-10  
--- Товар: Олія соняшникова | Ціна: 55.00 | Залишок: 20 л | Завезено: 2024-01-2  
--- Товар: Масло вершкове | Ціна: 180.00 | Залишок: 15 кг | Завезено: 2024-01-1  
--- Товар: Сир твердий | Ціна: 220.00 | Залишок: 12 кг | Завезено: 2024-01-21  
--- Товар: Ковбаса варена | Ціна: 150.00 | Залишок: 18 кг | Завезено: 2024-01-2  
--- Товар: Помідори | Ціна: 60.00 | Залишок: 40 кг | Завезено: 2024-01-23  
--- Товар: Огірки | Ціна: 45.00 | Залишок: 35 кг | Завезено: 2024-01-23  
--- Товар: Картопля | Ціна: 20.00 | Залишок: 100 кг | Завезено: 2024-01-15  
--- Товар: Цибуля ріпчаста | Ціна: 25.00 | Залишок: 50 кг | Завезено: 2024-01-1  
--- Товар: Морква | Ціна: 30.00 | Залишок: 45 кг | Завезено: 2024-01-17
```

Рисунок 16 – відфільтрований товар

Обравши пункт 8 головного меню, консоль виводить інверталізацію залишків і загальну ціну залишку (рис. 17).

```
===== ІНВЕНТАРИЗАЦІЯ ЗАЛИШКІВ =====
```

Товар	Залишок	Ціна	Вартість
Хліб білий	50	25.50	1275.00
Молоко 1л	30	45.00	1350.00
Яйця курячі	120	3.50	420.00
Цукор білий	25	35.00	875.00
Олія соняшникова	20	55.00	1100.00
Масло вершкове	15	180.00	2700.00
Сир твердий	12	220.00	2640.00
Ковбаса варена	18	150.00	2700.00
Помідори	40	60.00	2400.00
Огірки	35	45.00	1575.00
Картопля	100	20.00	2000.00
Цибуля ріпчаста	50	25.00	1250.00
Морква	45	30.00	1350.00

СУМАРНА ВАРТІСТЬ ЗАЛИШКІВ:		21635.00 грн.	

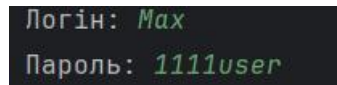
Рисунок 17 – інвентаризація залишків

Обравши пункт 9 головного меню, консоль виводить меню керування користувачами (рис. 18).

```
=== МЕНЮ КЕРУВАННЯ КОРИСТУВАЧАМИ ===
1. Переглянути всіх користувачів
2. Створити нового користувача
3. Видалити користувача
0. Повернутися до головного меню
Ваш вибір:
```

Рисунок 18 – керування користувачами

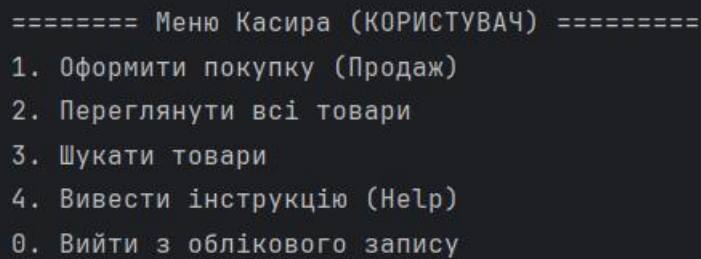
Після запуску програми, в консоль виводиться меню авторизації користувача, авторизовуємось як користувач (рис. 19).



```
Логін: Max  
Пароль: 1111user
```

Рисунок 19 – меню авторизації (користувача)

Після успішної авторизації як user ми бачимо меню, яке складається з 5 пунктів (рис. 20). Користувач може оформити покупку, переглянути всі товари, шукати товари, вивести інструкцію (Help).



```
===== Меню Касира (КОРИСТУВАЧ) =====  
1. Оформити покупку (Продаж)  
2. Переглянути всі товари  
3. Шукати товари  
4. Вивести інструкцію (Help)  
0. Вийти з облікового запису
```

Рисунок 20 – меню користувача

Вибравши пункт 1, консоль виводить оформлення покупки (ЧЕК №...) водиться назва, кількість товару (рис.21). Виводиться повний чек про покупку (рис. 22).

```

===== ОФОРМЛЕННЯ ПОКУПКИ (ЧЕК #1001) =====

Введіть назву товару (або '0' для завершення): 0гiрки
0гiрки
Введіть кількість для продажу: 3
// Успіх: Додано 3 од. 0гiрки до чека.

Введіть назву товару (або '0' для завершення): 0
0

```

Рисунок 21 – оформлення покупки

```

=====
МАГАЗИН: СИСТЕМА УПРАВЛІННЯ
Чек #1001                               | Дата: 2025-11-21 17:03:51
Касир: Мах
-----
Товар          ЦінаК-ть    СУМА
-----
0гiрки          45.00      3      135.00
-----
ВСЬОГО ДО СПЛАТИ:                135.00 грн
=====

```

Рисунок 22 – чек про покупку

Вибравши пункт 2, консоль виводить список товарів (рис. 23).

```
===== СПИСОК ТОВАРІВ =====
```

Назва	Ціна	Залишок	Од.	Дата Завозу
Хліб білий	25.50	50	шт	2024-01-15
Молоко 1л	45.00	30	л	2024-01-20
Яйця курячі	3.50	120	шт	2024-01-18
Цукор білий	35.00	25	кг	2024-01-10
Олія соняшникова	55.00	20	л	2024-01-22
Масло вершкове	180.00	15	кг	2024-01-19
Сир твердий	220.00	12	кг	2024-01-21
Ковбаса варена	150.00	18	кг	2024-01-20
Помідори	60.00	40	кг	2024-01-23
Огірки	45.00	32	кг	2024-01-23
Картопля	20.00	100	кг	2024-01-15
Цибуля ріпчаста	25.00	50	кг	2024-01-16
Морква	30.00	45	кг	2024-01-17

```
=====
```

Рисунок 23 – список товарів

Вибравши пункт 3, консоль виводить результат пошуку товару (рис. 24).

```
===== РЕЗУЛЬТАТИ ПОШУКУ за 'Помідор' =====
```

```
--- Товар: Помідори      | Ціна: 60.00 | Залишок: 40 кг | Завезено: 2024-01-23
```

Рисунок 24 – результат пошуку товару

Вибравши пункт 4, консоль виводить результат інструкції користувача (Help) (рис. 25).

```
===== ІНСТРУКЦІЯ КОРИСТУВАЧА (HELP) =====  
1. ПОЯСНЕННЯ РОБОТИ ПРОГРАМИ:  
  - Програма є консольним додатком для управління базою товарів магазину.  
  - Робота починається з авторизації (Admin має повні права, User - обмежені.  
  - Дані про товари зберігаються у файлі 'store_items.csv', дані користувачів - 'users.txt'.  
  - При виході дані автоматично зберігаються .  
  
2. ПРАВИЛА ВВОДУ ДАНИХ:  
  - Усі числові поля (ціна, кількість) вимагають введення додатних чисел.  
  - Система автоматично перевіряє коректність формату та діапазону введених даних.  
  
3. КОРОТКИЙ ОПИС КОМАНД (Касир):  
  - [1] Оформити покупку: Відкриває цикл продажу, виписує чек та коректує базу.  
  - [2] Переглянути: Виводить повний список товарів.  
  - [3] Шукати: Дозволяє знайти товар за частиною назви.  
  - [0] Вийти: Вихід з облікового запису.  
=====
```

Рисунок 25 – інструкція користувача

3 ВИСНОВКИ

У результаті виконання курсового проєкту розроблена система управління базою автомобілів для комп'ютерів, які працюють під керуванням ОС Windows.

У розробленому програмному забезпеченні передбачено:

- 1) Подання даних користувачем у консолі.
- 2) Збереження даних у файл.
- 3) Зчитування даних з файлу.
- 4) Додавання нових товарів.
- 5) Редагування доданих товарів.
- 6) Видалення товарів.
- 7) Перегляд всіх товарів.
- 8) Пошук товарів.
- 9) Сортування товарів.
- 10) Фільтрування товарів.
- 11) Керування обліковими записами.
- 12) Оформлення покупки.
- 13) Виведення інструкції (Help).

Дана розробка у майбутньому може бути розширена із додаванням нового функціоналу і видозміненою логікою обробки.

4 СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Bjarne Stroustrup. The C++ Programming Language (4th Edition). Addison-Wesley, 2013.
2. Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
3. ISO/IEC 14882. *International Standard for C++ (C++17 або C++20)*.
4. <https://www.tutorialspoint.com/cplusplus/index.htm>
5. <https://www.jetbrains.com/clion/features/code-documentation.html>
6. <https://www.jetbrains.com/clion/features/code-documentation.html>

5 ДОДАТКИ

ДОДАТОК А. Скролінг (текст) програми

Програмний модуль “Main”

Вміст “Main.cpp”

```
#include <complex>

#include "AuthManager.h"
#include "StoreManager.h"
#include "Configuration.h"
#include "SaleProcessor.h"
#include <iostream>
#include <limits>

int main()
{
    //ініціалізація основних менеджерів
    AuthManager authManager(Configuration::USER_DATA_FILE);
    StoreManager storeManager(Configuration::PRODUCT_DATA_FILE);
    SaleProcessor saleProcessor(storeManager);

    User* currentUser = nullptr;
    std::string login, password;
    int choice = -1;

    while (true)
    {
        if (!currentUser) {
            std::cout << "\n===== СИСТЕМА УПРАВЛІННЯ МАГАЗИНОМ =====" << std::endl;
            std::cout << "Введіть 0 для виходу або дані для авторизації." << std::endl;
            std::cout << "Логін: ";
            std::cin >> login;
            if (login == "0") break;
            std::cout << "Пароль: ";
            std::cin >> password;
            Configuration::ClearInputBuffer();

            if (authManager.AuthenticateUser(login, password)) {
                currentUser = authManager.GetCurrentUser();
            } else {
                continue;
            }
        }

        currentUser->DisplayMenu();
    }
}
```

```

std::cout << "Ваш вибір: ";

if (!(std::cin >> choice)) {
    std::cerr << "Некоректний ввід. Повторіть." << std::endl;
    Configuration::ClearInputBuffer();
    continue;
}
Configuration::ClearInputBuffer();

if (currentUser->IsAdmin()) {
    //обробка меню Адміністратора
    switch (choice) {
        case 1: storeManager.AddObject(); break;
        case 2: storeManager.EditObject(); break;
        case 3: storeManager.DeleteObject(); break;
        case 4: storeManager.ViewObjects(); break;
        case 5: storeManager.SearchObjects(); break;
        case 6: storeManager.SortObjects(); break;
        case 7: storeManager.FilterObjects(); break;
        case 8: storeManager.PerformInventory(); break;
        case 9: authManager.AdminUsersMenu(); break;
        case 0: authManager.Logout(); currentUser = nullptr; break;
        default: std::cerr << "Невірний вибір." << std::endl; break;
    }
} else {
    //обробка меню Користувача
    switch (choice) {
        case 1: saleProcessor.ProcessFullSale(currentUser->GetLogin()); break;
        case 2: storeManager.ViewObjects(); break;
        case 3: storeManager.SearchObjects(); break;
        case 4: storeManager.DisplayHelp(); break; std:
        case 0: authManager.Logout(); currentUser = nullptr; break;

        default: std::cerr << "Невірний вибір." << std::endl; break;
    }
}
}

std::cout << "Програма завершила роботу. Дані збережено." << std::endl;
return 0;
}

```


Програмний модуль “Admin”

Вміст “Admin.h”

```
#pragma once
#include "User.h"

class Admin final : public User
{
public:
    Admin(const std::string& login, const std::string& password)
        : User(login, password) {}

    bool IsAdmin() const override { return true; }

    void DisplayMenu() const override;
};
```

Програмний модуль “Admin”

Вміст “Admin.cpp”

```
#include "Admin.h"
#include <iostream>

void Admin::DisplayMenu() const
{
    std::cout << "\n=== Меню Адміністратора (АДМІН) ===" << std::endl;
    std::cout << "1. Додати новий товар" << std::endl;
    std::cout << "2. Редагувати товар" << std::endl;
    std::cout << "3. Видалити товар (Уцінка/Списання)" << std::endl;
    std::cout << "4. Переглянути всі товари" << std::endl;
    std::cout << "5. Шукати товари" << std::endl;
    std::cout << "6. Сортувати товари" << std::endl;
    std::cout << "7. Фільтрувати товари" << std::endl;
    std::cout << "8. Провести інвентаризацію залишків" << std::endl;
    std::cout << "--- Адміністрування користувачів ---" << std::endl;
    std::cout << "9. Керування обліковими записами" << std::endl;
    std::cout << "0. Вийти з облікового запису" << std::endl;
}
```

Програмний модуль “AuthManager”

Вміст “AuthManager.h”

```
#pragma once

#include "User.h"
#include <vector>
#include <memory>
#include <string>

class AuthManager final
{
public:
    AuthManager(const std::string& userFile);
    ~AuthManager();

    bool AuthenticateUser(const std::string& login, const std::string& password);
    void Logout() { m_currentUser.reset(); }

    void AddNewUser();
    void DeleteUser();
    void DisplayAllUsers() const;
    void AdminUsersMenu();

    User* GetCurrentUser() const { return m_currentUser.get(); }

private:
    std::vector<std::unique_ptr<User>> m_users;
    std::unique_ptr<User> m_currentUser;
    const std::string m_userFile;

    bool LoadUsersFromFile();
    void SaveUsers() const;

    bool ValidateUserCredentials(const std::string& login, const std::string& password) const;
};
```

Програмний модуль “AuthManager”

Вміст “AuthManager.cpp”

```

#include "AuthManager.h"
#include "Configuration.h"
#include "Admin.h"
#include "BasicUser.h"
#include <fstream>
#include <sstream>
#include <iostream>
#include <algorithm>
#include <limits>
#include <iomanip>
#include <stdexcept>

AuthManager::AuthManager(const std::string& userFile) : m_userFile(userFile)
{
    Configuration::s_logger->LogInfo("Ініціалізація AuthManager.");
    LoadUsersFromFile();
}

AuthManager::~AuthManager()
{
    SaveUsers();
    Configuration::s_logger->LogInfo("AuthManager знищено. Дані користувачів збережено.");
}

bool AuthManager::LoadUsersFromFile()
{
    {
        std::ifstream file(m_userFile);
        if (!file.is_open()) {
            Configuration::s_logger->LogInfo("Файл користувачів не знайдено. Створення базового Admin.");
            std::cout << " // Попередження: Файл користувачів не знайдено. Створення базового Admin." << std::endl;
            m_users.push_back(std::make_unique<Admin>("admin", "admin123"));
            return false;
        }

        std::string line;
        while (std::getline(file, line)) {
            size_t colonPos = line.find(':');

            try {
                if (colonPos != std::string::npos) {
                    std::string login = line.substr(0, colonPos);
                    std::string password = line.substr(colonPos + 1);

                    bool isAdmin = (login == "admin");

                    if (isAdmin) {
                        m_users.push_back(std::make_unique<Admin>(login, password));
                    }
                }
            } catch (...) {
                Configuration::s_logger->LogInfo("Помилка при завантаженні користувачів з файлу.");
            }
        }
    }
}

```

```

    } else {
        m_users.push_back(std::make_unique<BasicUser>(login, password));
    }
} else {
    throw std::runtime_error("Невірний формат рядка (відсутній ':')");
}
} catch (const std::exception& e) {
    Configuration::s_logger->LogError("Помилка формату даних у users.txt: " + line + " (" + e.what() + ")");
    std::cerr << "// Помилка формату даних у users.txt: " << e.what() << std::endl;
}
}
Configuration::s_logger->LogInfo("Користувачів успішно завантажено: " + std::to_string(m_users.size()));
return true;
}

void AuthManager::SaveUsers() const
{
    std::ofstream file(m_userFile);
    if (!file.is_open()) {
        Configuration::s_logger->LogError("Не вдалося зберегти дані користувачів.");
        std::cerr << "// Помилка: Не вдалося зберегти дані користувачів." << std::endl;
        return;
    }

    for (const auto& user : m_users) {
        file << user->GetLogin() << ":" << user->GetPassword() << std::endl;
    }
}

bool AuthManager::ValidateUserCredentials(const std::string& login, const std::string& password) const
{
    if (login.empty() || password.empty()) {
        Configuration::s_logger->LogError("Спроба авторизації з порожніми логіном/паролем.");
        std::cerr << "// Помилка: Логін або пароль не можуть бути порожніми." << std::endl;
        return false;
    }
    return true;
}

bool AuthManager::AuthenticateUser(const std::string& login, const std::string& password)
{
    if (!ValidateUserCredentials(login, password)) return false;

    auto it = std::find_if(m_users.begin(), m_users.end(),
        [&](const auto& user) {
            return user->GetLogin() == login && user->GetPassword() == password;
        });

    if (it != m_users.end()) {
        if ((*it)->IsAdmin()) {
            m_currentUser = std::make_unique<Admin>(login, password);
        } else {
            m_currentUser = std::make_unique<BasicUser>(login, password);
        }
    }
}

```

```

    }
    m_currentUser->SetAuthenticated(true);
    Configuration::s_logger->LogInfo("Успішна авторизація користувача: " + login + (m_currentUser->IsAdmin() ? " (Admin)" : " (User)")); // Журналювання
    std::cout << " // Авторизація успішна. Вітаємо, " << login << "!" << std::endl;
    return true;
}

Configuration::s_logger->LogError("Невірна спроба авторизації для логіна: " + login);
std::cerr << " // Помилка: Невірний логін або пароль." << std::endl;
return false;
}

void AuthManager::AddNewUser()
{
    std::string login, password, roleStr;
    std::cout << "--- СТВОРЕННЯ КОРИСТУВАЧА ---" << std::endl;
    std::cout << "Введіть новий логін: ";
    Configuration::ClearInputBuffer();
    std::getline(std::cin, login);

    if (std::any_of(m_users.begin(), m_users.end(), [&](const auto& u){ return u->GetLogin() == login; }))) {
        Configuration::s_logger->LogError("Спроба додати існуючого користувача: " + login); // Журналювання
        std::cerr << " // Помилка: Користувач з логіном " << login << " вже існує." << std::endl;
        return;
    }

    std::cout << "Введіть пароль: ";
    std::getline(std::cin, password);

    std::cout << "Введіть роль (admin/user): ";
    std::getline(std::cin, roleStr);
    std::transform(roleStr.begin(), roleStr.end(), roleStr.begin(), ::tolower);

    bool isAdmin = (roleStr == "admin");

    if (isAdmin) {
        m_users.push_back(std::make_unique<Admin>(login, password));
    } else {
        m_users.push_back(std::make_unique<BasicUser>(login, password));
    }

    Configuration::s_logger->LogInfo("Створено нового користувача: " + login + (isAdmin ? " (Admin)" : " (User)")); // Журналювання
    std::cout << " // Користувач " << login << " (" << (isAdmin ? "Admin" : "User") << ") успішно доданий." << std::endl;
}

void AuthManager::DeleteUser()
{
    std::string login;
    std::cout << "--- ВИДАЛЕННЯ КОРИСТУВАЧА ---" << std::endl;
    std::cout << "Введіть логін для видалення: ";
    Configuration::ClearInputBuffer();
    std::getline(std::cin, login);

```

```

if (m_currentUser && m_currentUser->GetLogin() == login) {
    Configuration::s_logger->LogError("Спроба видалити власний обліковий запис: " + login); // Журналювання
    std::cerr << "// Помилка: Не можна видалити власний обліковий запис." << std::endl;
    return;
}

auto oldSize = m_users.size();

m_users.erase(std::remove_if(m_users.begin(), m_users.end(),
    [&](const auto& u) { return u->GetLogin() == login; }), m_users.end());

if (m_users.size() < oldSize) {
    Configuration::s_logger->LogInfo("Користувача видалено: " + login); // Журналювання
    std::cout << "// Користувач '" << login << "' успішно видалений." << std::endl;
} else {
    Configuration::s_logger->LogError("Спроба видалити неіснуючого користувача: " + login); // Журналювання
    std::cerr << "// Помилка: Користувач '" << login << "' не знайдений." << std::endl;
}
}

void AuthManager::DisplayAllUsers() const
{
    std::cout << "\n===== ЗАПЕЄСТРОВАНИ КОРИСТУВАЧІ =====" << std::endl;
    std::cout << std::left << std::setw(20) << "ЛОГІН" << "|"
        << std::left << std::setw(15) << "ПАРОЛЬ (ВІДКРИТО)" << "|"
        << "РОЛЬ" << std::endl;
    std::cout << "-----|-----|-----" << std::endl;

    for (const auto& user : m_users) {
        std::cout << std::left << std::setw(20) << user->GetLogin() << "|"
            << std::left << std::setw(15) << user->GetPassword() << "|"
            << (user->IsAdmin() ? "Admin" : "User") << std::endl;
    }
    std::cout << "===== " << std::endl;
}

void AuthManager::AdminUsersMenu()
{
    int choice;
    do {
        std::cout << "\n=== МЕНЮ КЕРУВАННЯ КОРИСТУВАЧАМИ ===" << std::endl;
        std::cout << "1. Переглянути всіх користувачів" << std::endl;
        std::cout << "2. Створити нового користувача" << std::endl;
        std::cout << "3. Видалити користувача" << std::endl;
        std::cout << "0. Повернутися до головного меню" << std::endl;
        std::cout << "Ваш вибір: ";

        if (!(std::cin >> choice)) {
            Configuration::s_logger->LogError("Некоректний ввід у меню керування користувачами."); // Журналювання
            std::cerr << "// Некоректний ввід." << std::endl;
            Configuration::ClearInputBuffer();
            continue;
        }
    } while (choice < 0 || choice > 3);
}

```

```
}  
Configuration::ClearInputBuffer();  
  
switch (choice) {  
    case 1: DisplayAllUsers(); break;  
    case 2: AddNewUser(); break;  
    case 3: DeleteUser(); break;  
    case 0: break;  
    default:  
        Configuration::s_logger->LogError("Невірний вибір AdminUsersMenu: " + std::to_string(choice)); // Журналювання  
        std::cerr << "Невірний вибір." << std::endl;  
        break;  
    }  
} while (choice != 0);  
}
```


Програмний модуль “BasicUser”

Вміст “BasicUser.h”

```
#pragma once

#include "User.h"

class BasicUser final : public User
{
public:
    BasicUser(const std::string& login, const std::string& password)
        : User(login, password) {}

    bool IsAdmin() const override { return false; }

    void DisplayMenu() const override;
};
```

Програмний модуль “BasicUser”

Вміст “BasicUser.cpp”

```
#include "BasicUser.h"
#include <iostream>

void BasicUser::DisplayMenu() const
{
    std::cout << "\n===== Меню Касира (КОРИСТУВАЧ) =====< std::endl;
    std::cout << "1. Оформити покупку (Продаж)" << std::endl;
    std::cout << "2. Переглянути всі товари" << std::endl;
    std::cout << "3. Шукати товари" << std::endl;
    std::cout << "4. Вивести інструкцію (Help)" << std::endl;
    std::cout << "0. Вийти з облікового запису" << std::endl;
}
```

Програмний модуль “Configuration”

Вміст “Configuration.h”

```
#pragma once

#include <string>
#include <iostream>
#include <limits>
#include "Logger.h"

class Configuration final
{
public:
    static const std::string PRODUCT_DATA_FILE;
    static const std::string USER_DATA_FILE;
    static Logger* s_logger;

    static void ClearInputBuffer();

    template <typename T>
    static T GetSafeNumberInput(const std::string& prompt);

private:
    Configuration() = delete;
};

template <typename T>
T Configuration::GetSafeNumberInput(const std::string& prompt)
{
    T value;
    std::cout << prompt;

    while (!(std::cin >> value) || value < 0)
    {
        std::cerr << "// Помилка: Некоректний ввід. Будь ласка, введіть додатне число: ";
        ClearInputBuffer();
    }
    ClearInputBuffer();

    return value;
}
```

Програмний модуль “BasicUser”

Вміст “BasicUser.cpp”

```
#include "Configuration.h"
#include <limits>

const std::string Configuration::PRODUCT_DATA_FILE = "store_items.csv";
const std::string Configuration::USER_DATA_FILE = "users.txt";

static Logger loggerInstance;
Logger* Configuration::s_logger = &loggerInstance;

void Configuration::ClearInputBuffer()
{
    std::cin.clear();
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
}
```

Програмний модуль “IPrintable”

Вміст “IPretable.h”

```
#pragma once

#include <iostream>

class IPrintable
{

public:
    virtual void DisplayInfo() const = 0;

    virtual ~IPrintable() = default;
};
```

Програмний модуль “Logger”

Вміст ”Logeer.h”

```
#pragma once

#include <string>
#include <iostream>

class Logger
{
public:
    void LogInfo(const std::string& message) const
    {
        std::cout << "[INFO] " << message << std::endl;
    }

    void LogError(const std::string& message) const
    {
        std::cerr << "[ERROR] " << message << std::endl;
    }
};
```

Програмний модуль “Product”

Вміст “Product.h”

```
#pragma once
#include "IPrintable.h"
#include <string>

class Product : public IPrintable
{
public:
    virtual std::string GetName() const = 0;
    virtual void SetName(const std::string& name) = 0;
    virtual double GetPrice() const = 0;
    virtual void SetPrice(double price) = 0;

    virtual std::string ToCSVString() const = 0;

    virtual void DisplayInfo() const override = 0;

    virtual ~Product() = default;
};
```

Програмний модуль “Receipt”

Вміст “Receipt.h”

```
#pragma once
#include "IPrintable.h"
#include <string>

class Product : public IPrintable
{
public:
    virtual std::string GetName() const = 0;
    virtual void SetName(const std::string& name) = 0;
    virtual double GetPrice() const = 0;
    virtual void SetPrice(double price) = 0;

    virtual std::string ToCSVString() const = 0;

    virtual void DisplayInfo() const override = 0;

    virtual ~Product() = default;
};
```


Програмний модуль “Receipt”

Вміст “Receipt.cpp”

```
#include "Receipt.h"

#include <iostream>
#include <iomanip>
#include <sstream>
#include <chrono>

Receipt::Receipt(int receiptId, const std::string& cashierName)
: m_receiptId(receiptId), m_cashierName(cashierName)
{
    auto now = std::chrono::system_clock::to_time_t(std::chrono::system_clock::now());
    std::stringstream ss;
    ss << std::put_time(std::localtime(&now), "%Y-%m-%d %H:%M:%S");
    m_timestamp = ss.str();
}

void Receipt::AddItem(const std::string& itemName, double price, int quantity)
{
    m_items.push_back({itemName, price, quantity});
}

double Receipt::CalculateTotal() const
{
    double total = 0.0;
    for (const auto& item : m_items) {
        total += item.lineTotal();
    }
    return total;
}

void Receipt::DisplayInfo() const
{
    std::cout << "\n===== " << std::endl;
    std::cout << std::left << std::setw(40) << "МАГАЗИН: СИСТЕМА УПРАВЛІННЯ" << std::endl;
    std::cout << "Чек #" << std::left << std::setw(30) << m_receiptId
        << " | Дата: " << m_timestamp << std::endl;
    std::cout << "Касир: " << m_cashierName << std::endl;
    std::cout << "-----" << std::endl;
    std::cout << std::left << std::setw(20) << "Товар" << std::right << std::setw(8) << "Ціна"
        << std::setw(5) << "К-ть" << std::setw(12) << "СУМА" << std::endl;
    std::cout << "-----" << std::endl;

    for (const auto& item : m_items) {
        std::cout << std::left << std::setw(20) << item.name
            << std::right << std::setw(8) << std::fixed << std::setprecision(2) << item.price
            << std::setw(5) << item.quantity
            << std::setw(12) << item.lineTotal() << std::endl;
    }
}
```

```

}

std::cout << "-----" << std::endl;
std::cout << std::left << std::setw(35) << "ВСЬОГО ДО СПЛАТИ:" << std::right << std::setw(14)
    << std::fixed << std::setprecision(2) << CalculateTotal() << " грн" << std::endl;
std::cout << "===== " << std::endl;
}

```

Програмний модуль “SaleProcessor”

Вміст “SaleProcessor.h”

```
#pragma once

#include <string>
#include <vector>
#include "IPrintable.h"

class Receipt final : public IPrintable
{
public:
    Receipt(int receiptId, const std::string& cashierName);

    struct ReceiptItem {
        std::string name;
        double price;
        int quantity;
        double lineTotal() const { return price * quantity; }
    };

    void AddItem(const std::string& itemName, double price, int quantity);
    double CalculateTotal() const;
    int GetReceiptId() const { return m_receiptId; }
    void DisplayInfo() const override;

private:
    int m_receiptId;
    std::string m_cashierName;
    std::string m_timestamp;
    std::vector<ReceiptItem> m_items;
};
```

Програмний модуль “SaleProcessor”

Вміст “SaleProcessor.cpp”

```

#include "SaleProcessor.h"
#include "StoreManager.h"
#include "Configuration.h"
#include <iostream>
#include <limits>
#include <iomanip>

SaleProcessor::SaleProcessor(StoreManager& manager)
    : m_storeManager(manager) {}

void SaleProcessor::ProcessFullSale(const std::string& cashierName)
{
    Receipt currentReceipt(getNextReceiptId(), cashierName);
    bool addingItems = true;

    std::cout << "\n===== ОФОРМЛЕННЯ ПОКУПКИ (ЧЕК #" << currentReceipt.GetReceiptId() << ") =====" <<
    std::endl;

    while (addingItems)
    {
        std::string name;
        int quantity;

        std::cout << "\nВведіть назву товару (або '0' для завершення): ";
        Configuration::ClearInputBuffer();
        std::getline(std::cin, name);

        if (name == "0") {
            addingItems = false;
            continue;
        }

        quantity = Configuration::GetSafeNumberInput<int>("Введіть кількість для продажу: ");

        if (quantity == 0) continue;

        StoreItem* item = m_storeManager.GetItemForSale(name);

        if (item) {
            if (item->GetQuantity() >= quantity) {
                currentReceipt.AddItem(item->GetName(), item->GetPrice(), quantity);
                item->UpdateQuantity(-quantity); // Коректування бази
                std::cout << "// Уснix: Додано " << quantity << " од. " << item->GetName() << " до чека." << std::endl;
            }
        }
    }
}

```

```

    } else {
        std::cerr << "// Помилка: Недостатньо " << item->GetName() << ". Залишок: " << item->GetQuantity() << " од." <<
std::endl;
    }
} else {
    std::cerr << "// Помилка: Товар " << name << " не знайдено у базі." << std::endl;
}
}

if (currentReceipt.CalculateTotal() > 0) {
    currentReceipt.DisplayInfo();
    std::cout << "\n// База наявності товарів скоректована згідно з продажем." << std::endl;
} else {
    std::cout << "// Покупку скасовано. Чек порожній." << std::endl;
}
}
}

```

Програмний модуль “StoreItem”

Вміст “StoreItem.h”

```
#pragma once

#include "Product.h"

class StoreItem final : public Product
{
public:
    // Конструктори та Деструктор
    StoreItem();
    StoreItem(const std::string& name, const std::string& unit, double price, int quantity, const std::string& lastSupplyDate);
    StoreItem(const StoreItem& other);
    ~StoreItem();

    std::string GetName() const override { return m_name; }
    void SetName(const std::string& name) override { m_name = name; }
    std::string GetUnit() const { return m_unit; }
    void SetUnit(const std::string& unit) { m_unit = unit; }
    double GetPrice() const override { return m_price; }
    void SetPrice(double price) override { m_price = price; }
    int GetQuantity() const { return m_quantity; }
    void SetQuantity(int quantity) { m_quantity = quantity; }
    void UpdateQuantity(int change) { m_quantity += change; }
    std::string GetLastSupplyDate() const { return m_lastSupplyDate; }
    void SetLastSupplyDate(const std::string& date) { m_lastSupplyDate = date; }

    void DisplayInfo() const override;
    std::string ToCSVString() const override;

private:
    std::string m_name;
    std::string m_unit;
    double m_price;
    int m_quantity;
    std::string m_lastSupplyDate;
};
```

Програмний модуль “StoreItem”

Вміст “StoreItem.cpp”

```
#include "StoreItem.h"

#include <sstream>
#include <iostream>
#include <iomanip>

StoreItem::StoreItem()
: m_name(""), m_unit(""), m_price(0.0), m_quantity(0), m_lastSupplyDate("") {}

StoreItem::StoreItem(const std::string& name, const std::string& unit, double price, int quantity, const std::string& lastSupplyDate)
: m_name(name), m_unit(unit), m_price(price), m_quantity(quantity), m_lastSupplyDate(lastSupplyDate) {}

StoreItem::StoreItem(const StoreItem& other)
: Product(other), m_name(other.m_name), m_unit(other.m_unit), m_price(other.m_price), m_quantity(other.m_quantity),
m_lastSupplyDate(other.m_lastSupplyDate) {}

StoreItem::~StoreItem() {}

void StoreItem::DisplayInfo() const
{
    std::cout << "--- Товар: " << std::left << std::setw(20) << m_name
    << " | Ціна: " << std::fixed << std::setprecision(2) << m_price
    << " | Залишок: " << m_quantity
    << " " << m_unit << " | Завезено: " << m_lastSupplyDate << std::endl;
}

std::string StoreItem::ToCSVString() const
{
    std::stringstream ss;
    ss << m_name << "," << m_unit << "," << m_price << "," << m_quantity << "," << m_lastSupplyDate;
    return ss.str();
}
```

Програмний модуль “StoreManager”

Вміст “StoreManager.h”

```
#pragma once

#include "StoreItem.h"
#include "Configuration.h"
#include <vector>
#include <memory>
#include <functional>

using ProductPointer = std::unique_ptr<Product>;
using ProductCollection = std::vector<ProductPointer>;

class StoreManager final
{
public:
    StoreManager(const std::string& productFile);
    ~StoreManager();

    void AddObject();
    void EditObject();
    void DeleteObject();
    void ViewObjects() const;

    void SearchObjects() const;
    void SortObjects();
    void FilterObjects() const;
    void PerformInventory() const;
    void DisplayHelp() const;

    StoreItem* GetItemForSale(const std::string& name);

private:
    ProductCollection m_products;
    const std::string m_productFile;

    bool LoadProducts();
    bool SaveProducts() const;

    StoreItem* FindStoreItemByName(const std::string& name);
    void InternalSort(std::function<bool(const ProductPointer&, const ProductPointer&)> comparator);
};
```


Програмний модуль “StoreManager”

Вміст “StoreManager.cpp”

```

#include "StoreManager.h"
#include "Configuration.h"
#include <fstream>
#include <sstream>
#include <algorithm>
#include <limits>
#include <iomanip>

StoreManager::StoreManager(const std::string& productFile) : m_productFile(productFile)
{
    LoadProducts();
}

StoreManager::~StoreManager()
{
    SaveProducts();
}

bool StoreManager::LoadProducts()
{
    {
        std::ifstream file(m_productFile);
        if (!file.is_open()) return false;

        std::string line;
        std::getline(file, line);

        while (std::getline(file, line)) {
            if (!line.empty() && line.back() == '\r') {
                line.pop_back();
            }

            // Пропускаємо порожні рядки
            if (line.empty()) continue;

            std::stringstream ss(line);
            std::string name, unit, lastSupplyDate, sPrice, sQuantity;

            try {
                if (!std::getline(ss, name, ',') ||
                    !std::getline(ss, unit, ',') ||
                    !std::getline(ss, sPrice, ',') ||
                    !std::getline(ss, sQuantity, ',') ||
                    !std::getline(ss, lastSupplyDate))
                {
                    std::cerr << "Помилка: Неповний рядок даних у файлі." << std::endl;
                }
            }
        }
    }
}

```

```

        continue;
    }

    double price = std::stod(sPrice);
    int quantity = std::stoi(sQuantity);

    if (price < 0 || quantity < 0) throw std::out_of_range("Negative value");

    m_products.push_back(std::make_unique<StoreItem>(name, unit, price, quantity, lastSupplyDate));

    } catch (const std::exception& e) {
        std::cerr << "// Помилка: Некоректні введення або формат даних у рядку: " << line << " (" << e.what() << ")" << std::endl;
    }
}

return true;
}

bool StoreManager::SaveProducts() const
{
    std::ofstream file(m_productFile);
    if (!file.is_open()) return false;

    file << "Name,Unit,Price,Quantity,LastSupplyDate" << std::endl;

    for (const auto& product : m_products) {
        file << product->ToCSVString() << std::endl;
    }

    return true;
}

StoreItem* StoreManager::FindStoreItemByName(const std::string& name)
{
    auto it = std::find_if(m_products.begin(), m_products.end(),
        [&name](const auto& p) { return p->GetName() == name; });

    if (it != m_products.end()) {
        return dynamic_cast<StoreItem*>(it->get());
    }

    return nullptr;
}

StoreItem* StoreManager::GetItemForSale(const std::string& name)
{
    return FindStoreItemByName(name);
}

void StoreManager::AddObject()
{
    std::string name, unit, lastSupplyDate;
    double price;
    int quantity;

    std::cout << "\n=== СТВОРЕННЯ НОВОГО ТОВАРУ ===" << std::endl;

```

```

std::cout << "Введіть назву товару: ";
Configuration::ClearInputBuffer();
std::getline(std::cin, name);

std::cout << "Введіть одиницю виміру (шт, кг, л): ";
std::getline(std::cin, unit);

price = Configuration::GetSafeNumberInput<double>("Введіть ціну одиниці: ");
quantity = Configuration::GetSafeNumberInput<int>("Введіть кількість: ");

std::cout << "Введіть дату останнього завозу (YYYYMMDD): ";
std::getline(std::cin, lastSupplyDate);

if (FindStoreItemByName(name) != nullptr) {
    std::cerr << "// Помилка: Товар з такою назвою вже існує. Операцію скасовано." << std::endl;
    return;
}

m_products.push_back(std::make_unique<StoreItem>(name, unit, price, quantity, lastSupplyDate));
std::cout << "// Товар " << name << " успішно додано до бази." << std::endl;
}

void StoreManager::EditObject()
{
    std::string name;
    std::cout << "\n=== РЕДАГУВАННЯ ТОВАРУ ===" << std::endl;
    std::cout << "Введіть назву товару для редагування: ";
    Configuration::ClearInputBuffer();
    std::getline(std::cin, name);

    StoreItem* item = FindStoreItemByName(name);

    if (item) {
        int choice;
        std::cout << "Що редагувати? 1. Ціна 2. Кількість 3. Одиниця виміру: ";
        if (!(std::cin >> choice)) {
            std::cerr << "// Невірний ввід." << std::endl;
            Configuration::ClearInputBuffer();
            return;
        }

        switch (choice) {
            case 1: {
                double newPrice = Configuration::GetSafeNumberInput<double>("Введіть нову ціну: ");
                item->SetPrice(newPrice);
                break;
            }
            case 2: {
                int newQuantity = Configuration::GetSafeNumberInput<int>("Введіть нову кількість: ");
                item->SetQuantity(newQuantity);
                break;
            }
        }
    }
}

```

```

    case 3: {
        std::string newUnit;
        std::cout << "Введіть нову одиницю виміру: ";
        Configuration::ClearInputBuffer();
        std::getline(std::cin, newUnit);
        item->SetUnit(newUnit);
        break;
    }
    default:
        std::cerr << "// Невірний вибір поля." << std::endl;
        return;
    }
    std::cout << "// Товар " << name << " успішно оновлено." << std::endl;
} else {
    std::cerr << "// Помилка: Товар не знайдено." << std::endl;
}
}

void StoreManager::DeleteObject()
{
    std::string name;
    std::cout << "\n=== ВИДАЛЕННЯ ТОВАРУ (Списання/Уцінка) ===" << std::endl;
    std::cout << "Введіть назву товару для видалення: ";
    Configuration::ClearInputBuffer();
    std::getline(std::cin, name);

    auto oldSize = m_products.size();

    m_products.erase(std::remove_if(m_products.begin(), m_products.end(),
        [&](const auto& p) { return p->GetName() == name; }), m_products.end());

    if (m_products.size() < oldSize) {
        std::cout << "// Товар " << name << " успішно списано/видалено з бази." << std::endl;
    } else {
        std::cerr << "// Помилка: Товар не знайдено." << std::endl;
    }
}

void StoreManager::ViewObjects() const
{
    if (m_products.empty()) {
        std::cout << "// Список товарів порожній." << std::endl;
        return;
    }

    std::cout << "\n===== СПИСОК ТОВАРІВ =====\n" << std::endl;
    std::cout << std::left << std::setw(22) << "Назва" << " | "
        << std::right << std::setw(9) << "Ціна" << " | "
        << std::right << std::setw(9) << "Залишок" << " | "
        << std::left << std::setw(5) << "Од." << " | "
        << std::left << "Дата Завозу" << std::endl;
    std::cout << "-----|-----|-----|----|-----" << std::endl;

```

```

for (const auto& product : m_products) {
    // Використовуємо dynamic_cast для доступу до полів StoreItem (якщо потрібно)
    if (auto item = dynamic_cast<StoreItem*>(product.get())) {
        std::cout << std::left << std::setw(22) << item->GetName() << " | "
            << std::right << std::setw(9) << std::fixed << std::setprecision(2) << item->GetPrice() << " | "
            << std::right << std::setw(9) << item->GetQuantity() << " | "
            << std::left << std::setw(5) << item->GetUnit() << " | "
            << std::left << item->GetLastSupplyDate() << std::endl;
    }
}

std::cout << "=====
" << std::endl;
}

void StoreManager::SearchObjects() const
{
    std::string criteria;
    std::cout << "Введіть частину назви для пошуку: ";
    Configuration::ClearInputBuffer();
    std::getline(std::cin, criteria);

    if (criteria.empty()) {
        std::cerr << "// Помилка: Критерій пошуку не може бути порожнім." << std::endl;
        return;
    }

    std::cout << "\n===== РЕЗУЛЬТАТИ ПОШУКУ за '" << criteria << "' =====" << std::endl;
    bool found = false;
    for (const auto& product : m_products) {
        if (product->GetName().find(criteria) != std::string::npos) {
            product->DisplayInfo();
            found = true;
        }
    }

    if (!found) {
        std::cout << "// Товарів, що відповідають критерію, не знайдено." << std::endl;
    }
}

void StoreManager::InternalSort(std::function<bool(const ProductPointer&, const ProductPointer&> comparator)
{
    std::sort(m_products.begin(), m_products.end(), comparator);
}

void StoreManager::SortObjects()
{
    int choice;
    std::cout << "Сортувати за: 1. Назва (ASC) 2. Ціна (ASC) 3. Кількість (DESC): ";
    if (!(std::cin >> choice)) {
        std::cerr << "// Невірний ввід." << std::endl;
        Configuration::ClearInputBuffer();
    }
}

```

```

    return;
}
Configuration::ClearInputBuffer();

switch (choice) {
    case 1:
        InternalSort([](const auto& a, const auto& b) { return a->GetName() < b->GetName(); });
        break;
    case 2:
        InternalSort([](const auto& a, const auto& b) { return a->GetPrice() < b->GetPrice(); });
        break;
    case 3:
        InternalSort([](const auto& a, const auto& b) {
            return dynamic_cast<StoreItem*>(a.get())->GetQuantity() >
                dynamic_cast<StoreItem*>(b.get())->GetQuantity();
        });
        break;
    default:
        std::cerr << "Невірний вибір." << std::endl;
        return;
}

std::cout << "// Сортювання виконано." << std::endl;
}

void StoreManager::FilterObjects() const
{
    int minQuantity;
    std::cout << "Показати товари, у яких залишок менший за: ";
    minQuantity = Configuration::GetSafeNumberInput<int>("");

    std::cout << "\n===== ФІЛЬТР: ЗАЛИШОК < " << minQuantity << " =====" << std::endl;
    bool found = false;
    for (const auto& product : m_products) {
        if (auto item = dynamic_cast<StoreItem*>(product.get())) {
            if (item->GetQuantity() < minQuantity) {
                item->DisplayInfo();
                found = true;
            }
        }
    }
    if (!found) {
        std::cout << "// Товарів, що відповідають критерію, не знайдено." << std::endl;
    }
}

void StoreManager::PerformInventory() const
{
    double totalValue = 0.0;

    std::cout << "\n===== ІНВЕНТАРИЗАЦІЯ ЗАЛИШКІВ =====" << std::endl;
    std::cout << std::left << std::setw(25) << "Товар" << "|" << std::right << std::setw(10) << "Залишок"
        << "|" << std::setw(10) << "Ціна" << "|" << std::setw(15) << "Вартість" << std::endl;

```

```

std::cout << "-----|-----|-----|-----" << std::endl;

for (const auto& product : m_products) {
    if (auto item = dynamic_cast<StoreItem*>(product.get())) {
        double productValue = item->GetPrice() * item->GetQuantity();
        totalValue += productValue;

        std::cout << std::left << std::setw(25) << item->GetName() << "|"
            << std::right << std::setw(10) << item->GetQuantity() << "|"
            << std::setw(10) << std::fixed << std::setprecision(2) << item->GetPrice() << "|"
            << std::setw(15) << productValue << std::endl;
    }
}

std::cout << "-----" << std::endl;
std::cout << std::left << std::setw(47) << "СУМАРНА ВАРТІСТЬ ЗАЛИШКІВ:" << std::right << std::setw(17)
    << std::fixed << std::setprecision(2) << totalValue << " грн." << std::endl;
}

void StoreManager::DisplayHelp() const
{
    std::cout << "\n===== ІНСТРУКЦІЯ КОРИСТУВАЧА (HELP) =====" << std::endl;

    std::cout << "1. ПОЯСНЕННЯ РОБОТИ ПРОГРАМИ:" << std::endl;
    std::cout << " - Програма є консольним додатком для управління базою товарів магазину." << std::endl;
    std::cout << " - Робота починається з авторизації (Admin має повні права, User - обмежені)." << std::endl;
    std::cout << " - Дані про товари зберігаються у файлі 'store_items.csv', дані користувачів - 'users.txt'." << std::endl;
    std::cout << " - При виході дані автоматично зберігаються." << std::endl;

    std::cout << "\n2. ПРАВИЛА ВВОДУ ДАНИХ:" << std::endl;
    std::cout << " - Усі числові поля (ціна, кількість) вимагають введення додатних чисел." << std::endl;
    std::cout << " - Система автоматично перевіряє коректність формату та діапазону введених даних." << std::endl;

    std::cout << "\n3. КОРОТКИЙ ОПИС КОМАНД (Касир):" << std::endl;
    std::cout << " - [1] Оформити покупку: Відкриває цикл продажу, випишує чек та коректує базу." << std::endl;
    std::cout << " - [2] Переглянути: Виводить повний список товарів." << std::endl;
    std::cout << " - [3] Шукати: Дозволяє знайти товар за частиною назви." << std::endl;
    std::cout << " - [0] Вийти: Вихід з облікового запису." << std::endl;

    std::cout << "===== " << std::endl;
    <<
    " << std::endl;
}

```

Програмний модуль “User”

Вміст “User.h”

```
#pragma once

#include <string>
#include <iostream>

class User
{
public:
    User(const std::string& login, const std::string& password);

    virtual ~User() = default;

    virtual bool IsAdmin() const = 0;
    virtual void DisplayMenu() const = 0;

    std::string GetLogin() const { return m_login; }
    std::string GetPassword() const { return m_password; }
    bool IsAuthenticated() const { return m_isAuthenticated; }
    void SetAuthenticated(bool status) { m_isAuthenticated = status; }

protected:
    std::string m_login;
    std::string m_password;
    bool m_isAuthenticated;
};
```


Програмный модуль “User”

Вміст User.cpp

```
#include "User.h"

User::User(const std::string& login, const std::string& password)
    : m_login(login), m_password(password), m_isAuthenticated(false)
{
}
```